

SOMMARIO

Classificazione di Immagini attraverso Reti convoluzionali	2
Presentazione problema.....	2
Dataset utilizzati	2
Pipeline adottata	2
Scelte progettuali.....	3
PreProcessing	4
Data Augmentation.....	4
Blocco Residuale custom	4
Iperparametri e motivazione delle scelte.....	5
Architettura completa	5
Numero di parametri	6
Risultati ottenuti in Validazione.....	7
Strategia 1	7
Strategia 2.....	8
Model selection e Risultati ottenuti in Test.....	8
Risultati sul test set del best model	9
Matrice di confusione	10
Possibili Miglioramenti	11
Architettura.....	11
Ensemble	11
Bibliografia.....	11
Testi e Risorse Utilizzate.....	12

CLASSIFICAZIONE DI IMMAGINI ATTRAVERSO RETI CONVOLUZIONALI

Assignment 3 del corso di Visione Artificiale, Ingegneria informatica - 2035 - LM-32

Studente: *Angelo Gulotta*, matricola 0830813

PRESENTAZIONE PROBLEMA

Data una immagine aerea in input, il sistema deve classificarla come appartenente ad una delle seguenti 21 classi: *agricultural, airplane, baseball_diamond, beach, buildings, chaparral, dense_residential, forest, freeway, golf_course, harbor, intersection, medium_residential, mobile_homepark, overpass, parking_lot, river, runway, sparse_residential, storage_tanks, tennis_court*.

DATASET UTILIZZATI

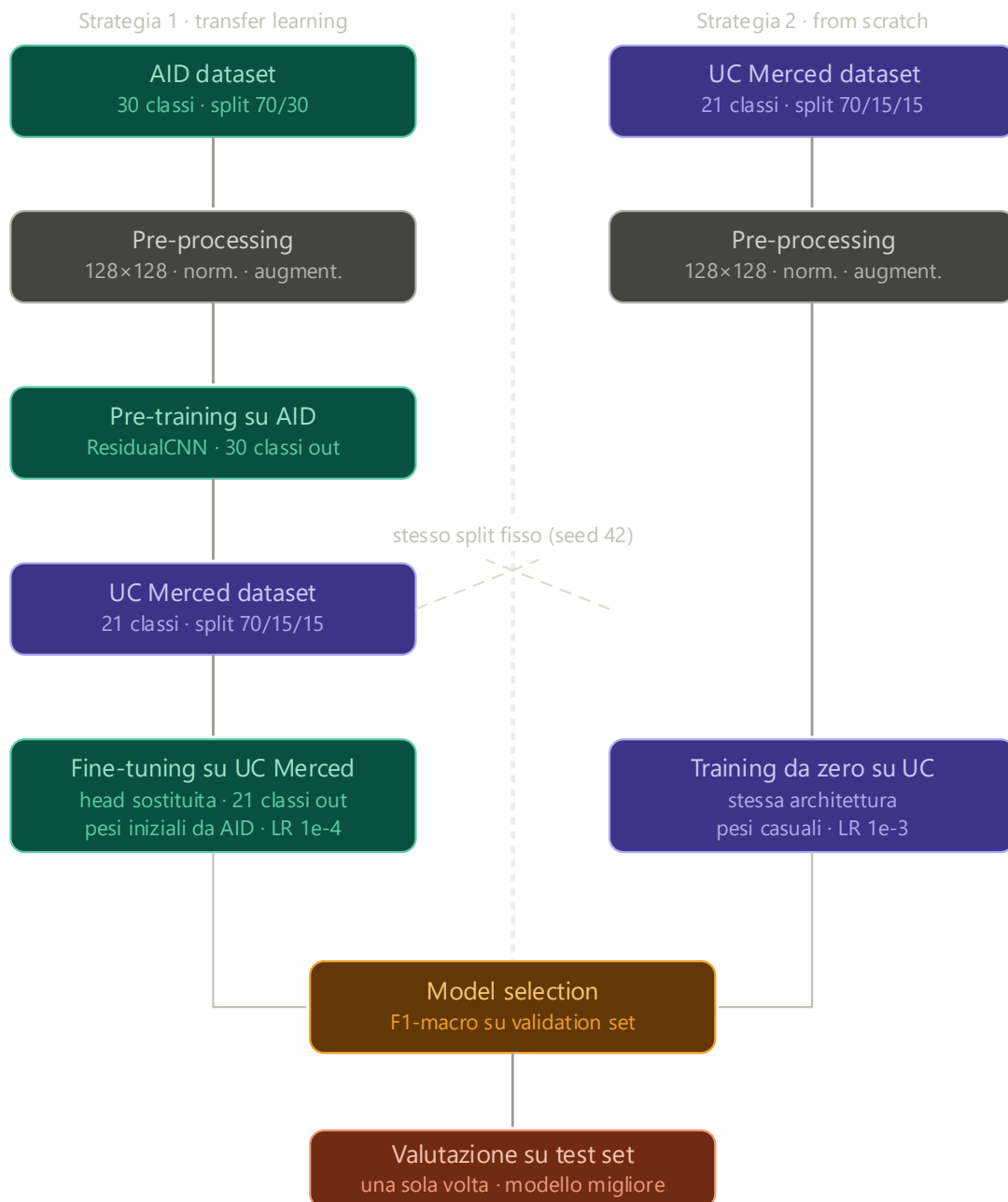
- **AID (Aerial Image Dataset)** è composto da circa 10.000 immagini suddivise in 30 classi di scene aeree, con un numero variabile di immagini per classe. Il pre-training è supervisionato: il modello viene addestrato a classificare correttamente le 30 classi di AID usando le etichette fornite dal dataset. L'obiettivo non è valutare le performance su AID ma apprendere feature generalizzabili da trasferire su UC Merced. Lo split adottato è 70% train e 30% validation, stratificato.
- **UC Merced Land Use** è il dataset target, composto da 2.100 immagini suddivise in 21 classi con esattamente 100 immagini per classe. Lo split è fisso e stratificato con seed 42: 1.470 immagini per il training, 315 per la validation e 315 per il test. Lo stesso split viene usato da entrambe le strategie, garantendo che il confronto sperimentale sia valido.

PIPELINE ADOTTATA

Il problema è affrontato tramite due strategie di addestramento indipendenti, confrontate sperimentalmente per valutare il contributo del transfer learning:

- **Strategia 1, Transfer learning:** Pre-training su AID (30 classi, ~10.000 immagini, split 70/30) seguito da fine-tuning su UC Merced (split 70/15/15 stratificato, seed fisso 42).
- **Strategia 2, Training da zero (baseline):** Stessa architettura, stessi dati UC Merced, stessi split. Pesi inizializzati casualmente, nessun trasferimento da AID.

Il confronto finale avviene sul **validation set** tramite F1-macro. Il test set viene utilizzato **una sola volta**, sul modello selezionato.



SCELTE PROGETTUALI

In questa sezione vengono riepilogate sinteticamente le decisioni progettuali principali, con un approfondimento delle scelte meno immediate.

PREPROCESSING

Il **preprocessing** è implementato in *dataset.py* tramite *OpenCV* ed è identico per entrambi i dataset. Ogni immagine viene caricata in BGR (formato nativo di *OpenCV*), convertita in RGB, **ridimensionata** a 128×128 pixel tramite *cv2.resize* e **normalizzata** in $[0, 1]$ dividendo per 255 e convertendo in float32. La **risoluzione 128×128** è stata scelta come compromesso tra qualità della rappresentazione e velocità di training su hardware consumer, rispettando il vincolo di restare sotto i 200×200 pixel imposto dall'assignment. Lo stesso identico preprocessing viene applicato in *inference.py* durante la predizione su nuove immagini.

DATA AUGMENTATION

La **data augmentation** viene applicata esclusivamente al **training set**, mai a validation e test. Le trasformazioni usate sono: flip orizzontale e verticale casuale, variazione casuale di luminosità con delta massimo di 0.15 e variazione casuale di contrasto nel range $[0.85, 1.15]$. Dopo l'augmentation viene applicato un clip in $[0, 1]$ per garantire che i valori rimangano nel range di normalizzazione.

BLOCCO RESIDUALE CUSTOM

Il blocco residuale implementa la formula $\mathbf{y} = \mathbf{F}(\mathbf{x}) + \mathbf{x}$

dove $\mathbf{F}(\mathbf{x})$ è il ramo principale e $\mathbf{x}$ è la skip connection.

Il blocco principale è composto da due layer convoluzionali in sequenza, questa scelta ha il vantaggio di aumentare la capacità espressiva del blocco senza appesantire il carico computazionale aumentando i parametri:

- Conv2D $3 \times 3 \rightarrow$ Batch Normalization $\rightarrow$ ReLU
- Conv2D $3 \times 3 \rightarrow$ Batch Normalization

In parallelo al blocco principale troviamo la **skip connection**, essa ha il compito di preservare il tensore di input per poi sommarlo alla fine del blocco con l'output del ramo principale:

- Identità se il numero di canali di input e output coincide
- Conv2D 1×1 + Batch Normalization se i canali cambiano (proiezione)

ReLU finale applicata dopo la somma $y = F(x) + x$

Il vantaggio principale di questa architettura riguarda il flusso del gradiente durante la backpropagation. In una rete senza skip connection il gradiente deve attraversare tutti i layer in sequenza e può diventare via via più piccolo fino ad azzerarsi (*vanishing gradient*). Con la skip connection la derivata di y rispetto a x vale $\partial y / \partial x = \partial F(x) / \partial x + 1$: il termine $+1$ garantisce che il gradiente non si azzeri mai, indipendentemente dalla profondità della rete.

IPERPARAMETRI E MOTIVAZIONE DELLE SCELTE

Scelta	Motivazione
Kernel 3x3	Cattura pattern locali (bordi, texture); standard nelle ResNet originali (He et al. 2016)
padding='same'	Mantiene le dimensioni spaziali costanti dentro il blocco, semplificando la somma con lo skip
use_bias=False	La Batch Normalization ha già un termine di bias implicito (parametro β); aggiungere un bias separato nella conv sarebbe ridondante
BN prima di ReLU	Scelta originale di He et al. 2016; normalizza la distribuzione prima dell'attivazione
Conv 1×1 per proiezione	Per allineare le dimensioni dello shortcut

ARCHITETTURA COMPLETA

La rete è organizzata in tre macro-componenti: uno stem iniziale, tre stage residuali con pooling, e un classificatore MLP finale.

Lo **stem** è un singolo layer Conv2D 3×3 con 32 filtri, seguito da Batch Normalization e ReLU. Ha il compito di produrre una prima rappresentazione dell'immagine di input (128×128×3) prima di entrare nei blocchi residuali.

I tre stage seguono tutti lo stesso schema: un blocco residuale seguito da un MaxPooling 2×2 che dimezza la risoluzione spaziale. Il numero di filtri raddoppia progressivamente (32, 64, 128): man mano che la risoluzione spaziale si riduce

aumenta la profondità semantica dei canali. Questa è la convenzione standard delle ResNet. Dopo i tre stage la feature map ha dimensione $16 \times 16 \times 128$.

Il classificatore MLP riceve in input il tensore appiattito tramite **Flatten**, che produce un vettore di $16 \times 16 \times 128 = 32.768$ neuroni. Seguono due layer Dense (256 e 128 neuroni) con ReLU e Dropout (0.4) ciascuno, e un layer di output Dense(21) con Softmax che produce una distribuzione di probabilità sulle 21 classi.

IPERPARAMETRI DI TRAINING

Parametro	Pre-Training AID	Fine-Tuning UC	Scratch UC
Optimizer	Adam	Adam	Adam
Learning rate	1e-3	1e-4	1e-3
Batch Size	32	32	32
Epoche Max	50	50	80
Early stopping patience	8	8	10

L'ottimizzatore scelto è **Adam** per tutte e tre le fasi. Adam è un ottimizzatore adattivo che aggiorna il learning rate individualmente per ogni parametro. È la scelta standard per il training di reti neurali profonde perché converge più rapidamente rispetto al SGD classico, soprattutto nelle fasi iniziali. Il learning rate del fine-tuning (1e-4) è 10 volte inferiore a quello del pre-training: un valore più alto sovrascriverebbe i pesi già appresi su AID, vanificando il trasferimento.

NUMERO DI PARAMETRI

Il modello conta in totale **8.733.173 parametri** in entrambe le configurazioni (fine-tuning e training da zero su UC Merced), di cui il backbone ne occupa solo 308.704 mentre il classificatore ne occupa circa 8.424.469. Lo squilibrio è causato dal Flatten: il vettore da 32.768 neuroni moltiplicato per i 256 neuroni del primo Dense produce da solo 8.388.608 pesi.

Strategia	Parametri Addestrabili	Parametri congelati
Frozen	2.709	8.730.464
Partial	8.654.613	78.560
Full	8.731.829	1.344

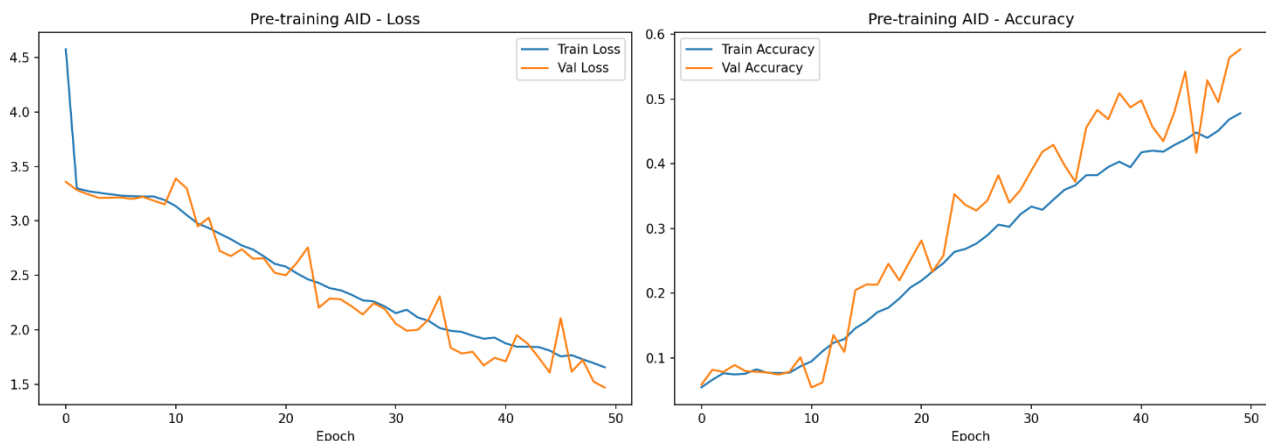
La strategia utilizzata nel fine-tuning è quella di congelare una parte dei pesi calcolati nella prima parte dell'addestramento, questa scelta è un compromesso tra preservare le feature di basso livello apprese su AID e adattare quelle di alto livello al dominio UC Merced. La strategia **partial** adottata congela il 60% iniziale dei layer, lasciando addestrabili gli ultimi layer convoluzionali e l'intero MLP; tuttavia, il learning rate ridotto evita di sovrascrivere i pesi appresi.

RISULTATI OTTENUTI IN VALIDAZIONE

Di seguito l'analisi dei risultati ottenuti in validazione che hanno portato alla scelta del modello della Strategia 1 come modello ottimale.

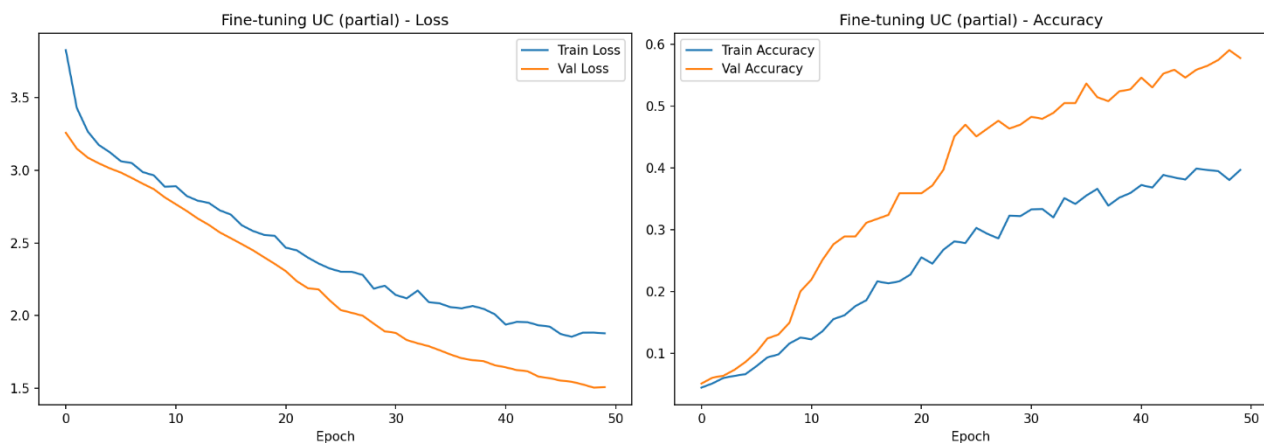
STRATEGIA 1

PRE-TRAINING



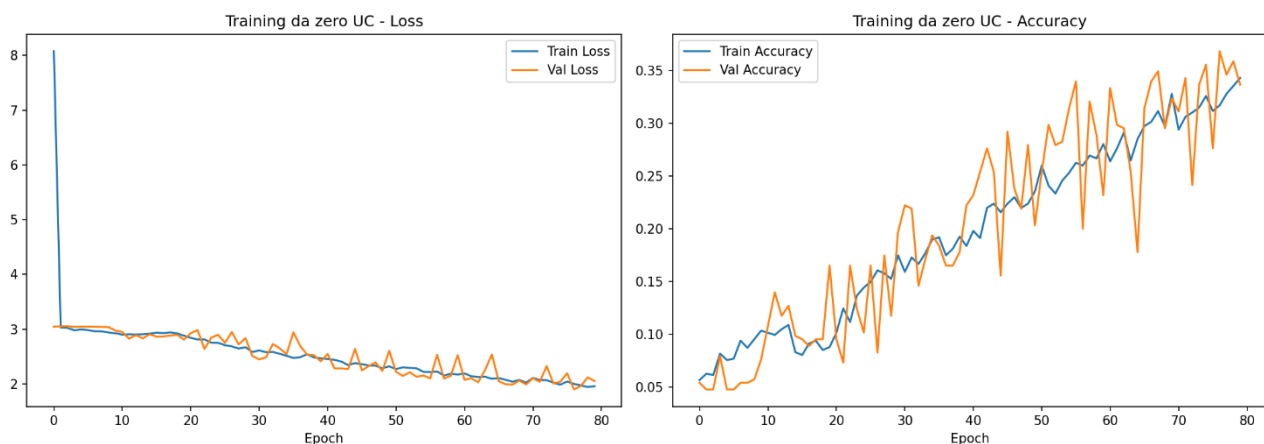
Il pre-training su AID completa tutte le 50 epoche senza che l'early stopping intervenga, il che indica che il modello era ancora in fase di apprendimento al termine del training. Sia la **training loss** che la **validation loss** scendono in modo costante per tutta la durata. La **validation loss** rimane sistematicamente inferiore alla **training loss**, comportamento atteso in presenza di **Dropout**: il Dropout è attivo solo durante il training, rendendo la rete artificialmente più debole nella fase di aggiornamento dei pesi rispetto alla fase di valutazione. L'accuracy finale si attesta attorno al 48% sul training set e al 58% sul validation set su 30 classi, un risultato ragionevole considerando la complessità del task e le dimensioni ridotte dell'architettura custom.

FINE-TUNING



Il **fine-tuning** completa anch'esso tutte le 50 epoche. La **val accuracy** cresce in modo costante lungo tutte le 50 epoche raggiungendo circa il 59%, mentre la training accuracy si attesta attorno al 40%. Questo comportamento è imputabile alla strategia **partial** con layer parzialmente congelati: i pesi già addestrati su AID forniscono immediatamente buone rappresentazioni al validation set, mentre il training set vede immagini aumentate che risultano più difficili da classificare. La val loss scende fino a circa 1.5. Anche qui val accuracy superiore a train accuracy è un effetto del Dropout, non un sintomo di data leakage.

STRATEGIA 2



Il training da zero completa tutte le 80 epoche. La rete mostra un apprendimento progressivo ma molto rumoroso: la val accuracy oscilla significativamente da un'epoca all'altra e raggiunge circa il 37% alla fine delle 80 epoche, con la val loss che scende da circa 3.0 a circa 2.0. L'instabilità è tipica del training con pochi dati e pesi inizializzati casualmente. Il distacco rispetto alla Strategia 1 rimane netto, confermando il valore del transfer learning su dataset di piccole dimensioni.

MODEL SELECTION E RISULTATI OTTENUTI IN TEST

La model selection viene effettuata confrontando le due strategie sul **validation set** tramite **F1-score macro**. Il test set non viene mai toccato in questa fase: usarlo per selezionare il modello introdurrebbe data leakage, rendendo la valutazione finale non rappresentativa del comportamento reale del sistema.

Il confronto è netto:

Strategia	F1-macro val
Strategia 1 (fine-tuning partial)	0.5516
Strategia 2 (training da zero)	0.3157

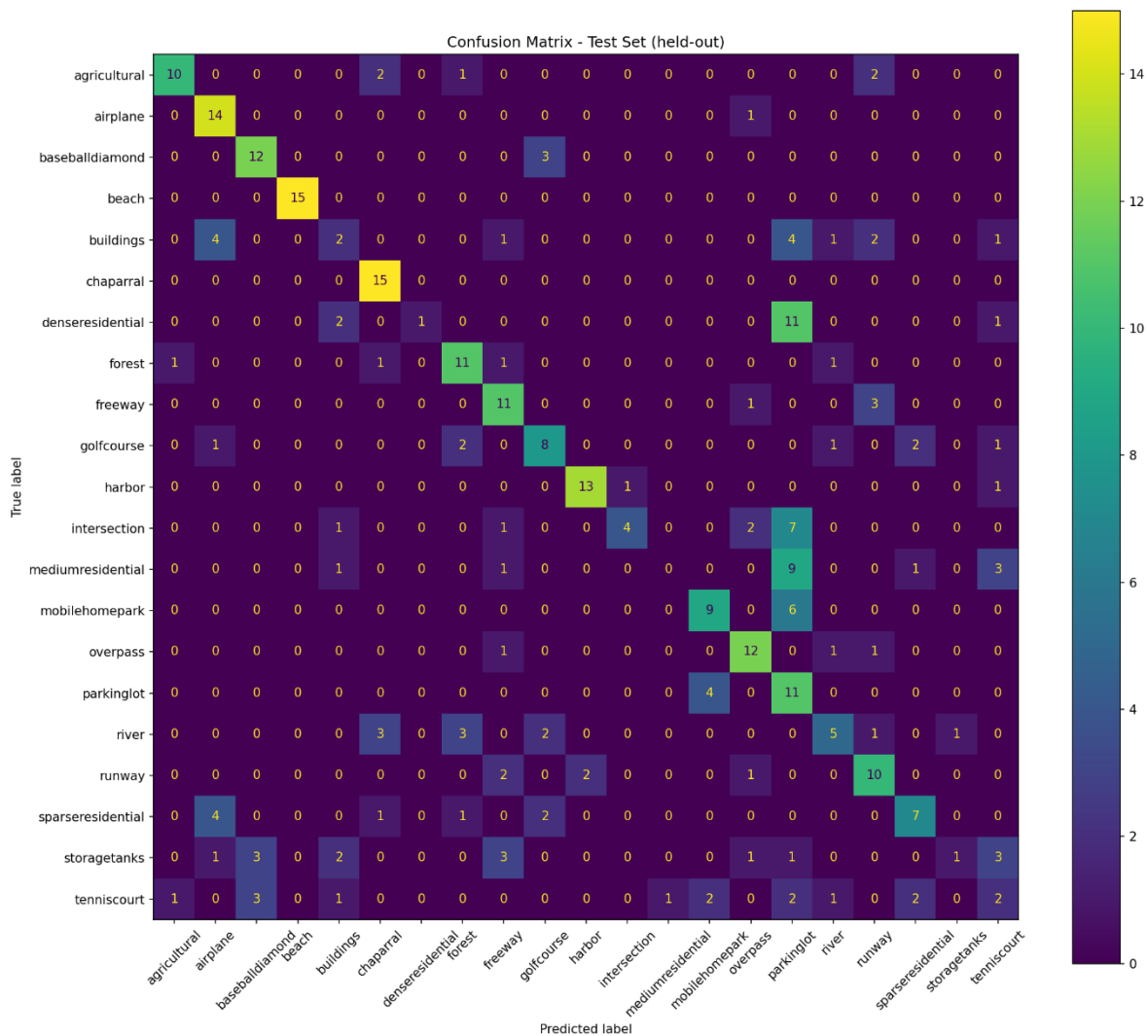
Il modello selezionato è quello della **Strategia 1**. Una volta selezionato, il test set viene utilizzato **una sola volta** per la valutazione finale.

RISULTATI SUL TEST SET DEL BEST MODEL

Metrica	Valore
Accuracy	54.9%
Macro Precision	57.5%
Macro Recall	54.9%
Macro F1	51.4%

Il modello selezionato ottiene sul test set una **accuracy** del 54.9% e un **F1-macro** del 51.4% su 21 classi. Il divario tra *precision* (57.5%) e *recall* (54.9%) indica che il modello tende a essere più cauto nelle predizioni: quando assegna una classe lo fa con discreta precisione, ma fatica a recuperare tutti i campioni di alcune classi difficili. Considerando che la baseline casuale su 21 classi sarebbe circa il 4.8%, il risultato conferma che il transfer learning ha permesso al modello di apprendere rappresentazioni utili nonostante il dataset di training limitato.

MATRICE DI CONFUSIONE



CLASSI CLASSIFICATE CORRETTAMENTE

Le classi con le performance migliori sono quelle con struttura visiva distintiva. *Beach* raggiunge il 100% di recall (15/15) grazie al colore uniforme della sabbia. *Chaparral* ottiene il 100% (15/15) per la texture arbustiva caratteristica. *Airplane* raggiunge il 93% (14/15) e *runway* il 67% (10/15). *Harbor* ottiene l'87% (13/15) grazie alla struttura portuale facilmente identificabile.

CLASSI CON PIÙ ERRORI

Medium residential ottiene 0% di recall (0/15): tutti i campioni vengono classificati erroneamente, principalmente come *overpass* e *mobile home park*. *Dense residential* raggiunge solo il 7% (1/15), confusa quasi sempre con *overpass*. *Storage tanks* ottiene il 7% (1/15), confusa con *buildings* e *baseball diamond*. *Buildings* ottiene il 13% (2/15), dispersa su molte classi diverse tra cui *airplane*, *freeway* e *parkinglot*.

CONFUSIONI RICORRENTI

Le confusioni più frequenti seguono una logica visiva precisa. *Medium residential* e *dense residential* vengono sistematicamente confuse con *overpass* e *mobile home park* per la similarità delle strutture viste dall'alto. *Storage tanks* viene confusa con *buildings* e *baseball diamond* per la forma circolare. *Parkinglot* genera molte predizioni errate su classi diverse, probabilmente perché la sua texture asfaltata uniforme ricorda altre superfici chiare.

POSSIBILI MIGLIORAMENTI

I risultati ottenuti, pur confermando il valore del transfer learning rispetto al training da zero, lasciano spazio a diversi miglioramenti sia architetturali che metodologici.

ARCHITETTURA

Una rete più profonda con più stage residuali permetterebbe di estrarre feature più ricche mantenendo un classificatore più leggero e riuscirebbe ad attenuare lo squilibrio tra backbone e classificatore.

ENSEMBLE

I due modelli prodotti dalle due strategie, pur con performance molto diverse, potrebbero essere combinati in un ensemble: la predizione finale sarebbe la media delle distribuzioni di probabilità dei due modelli. Anche se la Strategia 2 da sola è inutile, potrebbe correggere alcuni errori sistematici della Strategia 1 su specifiche classi.

BIBLIOGRAFIA

Durante lo sviluppo dell'assignment ho utilizzato come strumento di supporto alla progettazione il modello di intelligenza artificiale Gemini (Google) e Claude (Anthropic). L'impiego dell'AI mi ha consentito di velocizzare il processo di debugging, di ottimizzazione del progetto e della valutazione dei risultati.

Modalità di utilizzo:

- Discussione delle scelte progettuali e architetturali della pipeline
- Revisione e debugging degli script Python

- Supporto nella comprensione teorica di concetti quali vanishing gradient, transfer learning e strategie di fine-tuning

TESTI E RISORSE UTILIZZATE

Materiale Didattico del Corso: Slide e appunti forniti durante le lezioni.

He, K., Zhang, X., Ren, S., Sun, J. (2016). *Deep Residual Learning for Image Recognition*. CVPR 2016.

Data Augmentation — Deep Learning with TensorFlow, Ep. 19. Utilizzato come spunto per l'implementazione della data augmentation nella pipeline di training.

<https://www.youtube.com/watch?v=yke3zUGgQ-w>

Librerie Software

- TensorFlow 2.21.0 / Keras 3.12.1
- OpenCV 4.10.0.84
- scikit-learn 1.4.2
- NumPy 1.26.4
- Matplotlib 3.8.4